ДОМАШНЯЯ РАБОТА № 4

Отказоустойчивый расчёт навыков с Apache ZooKeeper

Кулинич Павел Васильевич
Инженерия данных 1.3
Вариант 5: «Программист, разработчик на PHP»

Результат проверки

Критерий	Фактический результат
Ансамбль	ZooKeeper 3.9.6; Java 17; лидер + 2 follower
Отказы серверов	2/3: запись успешна; 1/3: тайм-аут, кворума нет
Эфемерный узел	Создан настоящим zkCli; исчез после quit
FENCING=0	2 записи; версия указателя 2; владелец A
FENCING=1	1 запись; BadVersionError; версия 1; владелец B
Время смены лидера	Без защиты 11.268 с; с защитой 10.588 с
Совпадение с Д33	Все ключи и частоты при L=5/10/30; 12 словарей

1. Подготовка и ансамбль

Стенд поднят в трёх Docker-контейнерах. В официальном образе zookeeper:3.9.6 установлена OpenJDK 17.0.21. Вывод java -version, zkServer.sh version и status сохранён в logs/commands.txt. Установка Java в macOS для контейнерного варианта не требуется.

Сервер	myid	Порт на Mac	Роль при проверке
zk1	1	22181	follower
zk2	2	22182	leader
zk3	3	22183	follower

Конфигурация каждого сервера

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data
dataLogDir=/datalog
clientPort=2181
minSessionTimeout=10000
maxSessionTimeout=10000
4lw.commands.whitelist=ruok,srvr,stat,mntr
admin.enableServer=false
server.1=zk1:2888:3888
server.2=zk2:2888:3888
server.3=zk3:2888:3888
```

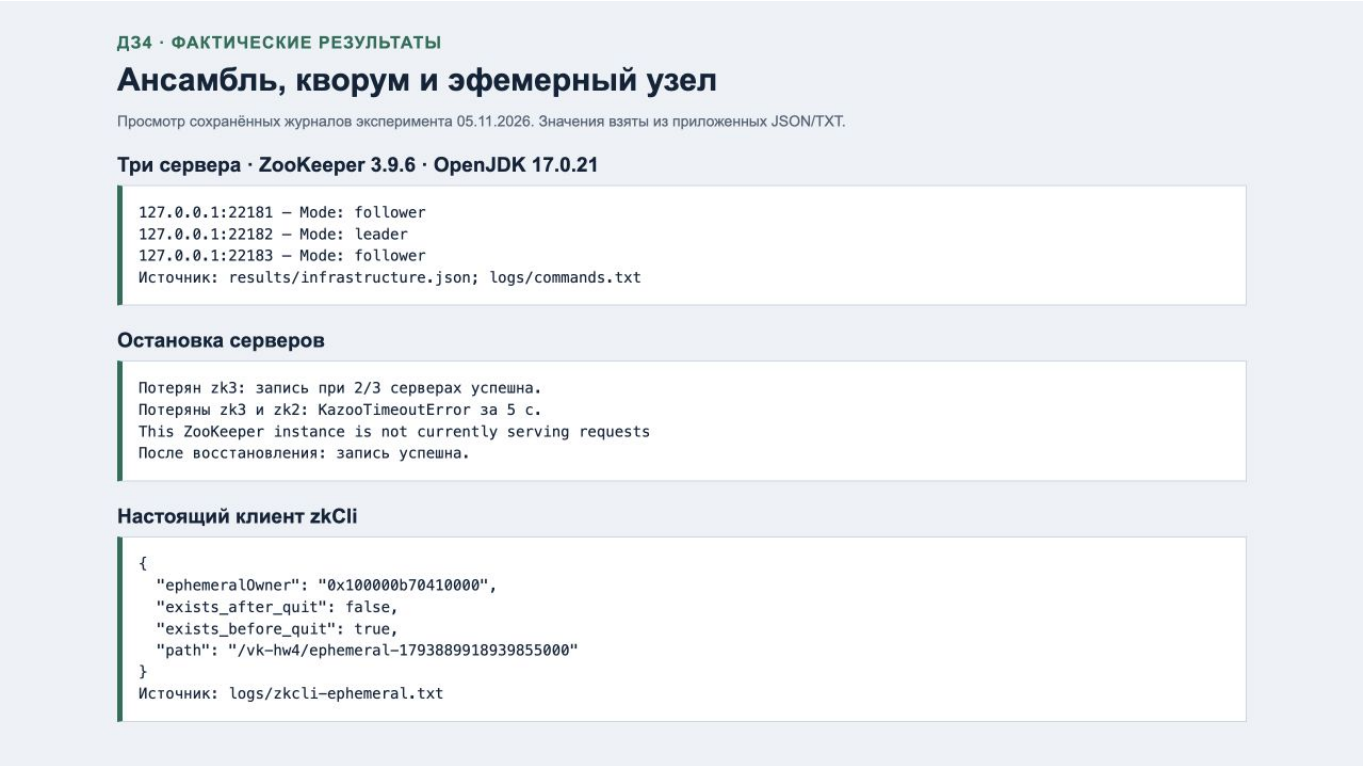
Строки server.N одинаковы. Файлы myid содержат 1, 2 и 3. Пути /data и /datalog внутри контейнеров одинаково называются, но каждый контейнер использует отдельные Docker volumes; пересечения данных нет. Внешние клиентские порты различаются. Изоляция контейнеров позволяет использовать внутри одинаковые 2181/2888/3888.

Команды запуска и наблюдения

```
docker compose -f infra/compose.yaml up -d
docker compose -f infra/compose.yaml exec -T zk1 zkServer.sh status
docker compose -f infra/compose.yaml exec -T zk2 zkServer.sh status
docker compose -f infra/compose.yaml exec -T zk3 zkServer.sh status
```

Образ зафиксирован тегом 3.9.6; фактический registry digest сохранён в logs/image-digest.txt. Параметры Compose: infra/compose.yaml.

2. Кворум и эфемерная сессия



ДЗ4 · ФАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ

Ансамбль, кворум и эфемерный узел

Просмотр сохранённых журналов эксперимента 05.11.2026. Значения взяты из приложенных JSON/TXT.

Три сервера · ZooKeeper 3.9.6 · OpenJDK 17.0.21

```
127.0.0.1:22181 — Mode: follower
127.0.0.1:22182 — Mode: leader
127.0.0.1:22183 — Mode: follower
Источник: results/infrastructure.json; logs/commands.txt
```

Остановка серверов

```
Потерян zk3: запись при 2/3 серверах успешна.
Потеряны zk3 и zk2: KazooTimeoutError за 5 с.
This ZooKeeper instance is not currently serving requests
После восстановления: запись успешна.
```

Настоящий клиент zkCli

```
{
  "ephemeralOwner": "0x100000b70410000",
  "exists_after_quit": false,
  "exists_before_quit": true,
  "path": "/vk-hw4/ephemeral-17938899189398550000"
}
Источник: logs/zkcli-ephemeral.txt
```

Скриншот браузера: локальная страница просмотра сохранённых результатов. Это представление фактических JSON/TXT, не снимок терминала. Полные журналы приложены.

В начале zk3 был лидером. После его остановки zk1 и zk2 сохранили большинство и выбрали нового лидера; реальная create-операция успешно завершилась. После остановки zk2 остался один сервер: create_async не завершился за 5 с (KazooTimeoutError), srvr сообщил, что сервер не обслуживает запросы. После запуска двух серверов новая запись снова прошла.

Тайм-аут означает отсутствие подтверждённой записи в наблюдаемом интервале, а не гарантированную отмену запроса: отложенный запрос может завершиться после восстановления кворума. В тесте он имеет отдельное уникальное имя и не влияет на расчёт.

Отдельный процесс zkCli создал узел командой create -e. Независимая Kazoo-сессия увидела ненулевой ephemeralOwner. После команды quit этот же наблюдатель подтвердил отсутствие узла. Вывод create/get -s/quit: logs/zkcli-ephemeral.txt; наблюдения: results/infrastructure.json.

3. Координатор и чекпоинт

Путь данных и координации

```
all_sources.csv -> batches(300) -> mapper(L=5/10/30)
                -> Counter -> snapshot JSON + SHA-256
ZooKeeper: /vk-hw4/<run>/
  election/     ephemeral sequential кандидаты Kazoo Lock
  epoch         монотонная эпоха; CAS-версия для fencing
  checkpoint    путь + SHA-256 + next_batch + epoch
  result        опубликованный указатель на результаты
```

Две копии запускают одинаковую программу. Kazoo Lock следит за предшественником в очереди кандидатов. Победитель увеличивает epoch условной записью и запоминает версию znode. Запасной получает лидерство после истечения сессии остановленного процесса. Запрошенный и фактически согласованный тайм-аут равен 10 000 мс; это записано в событии CONNECTED.

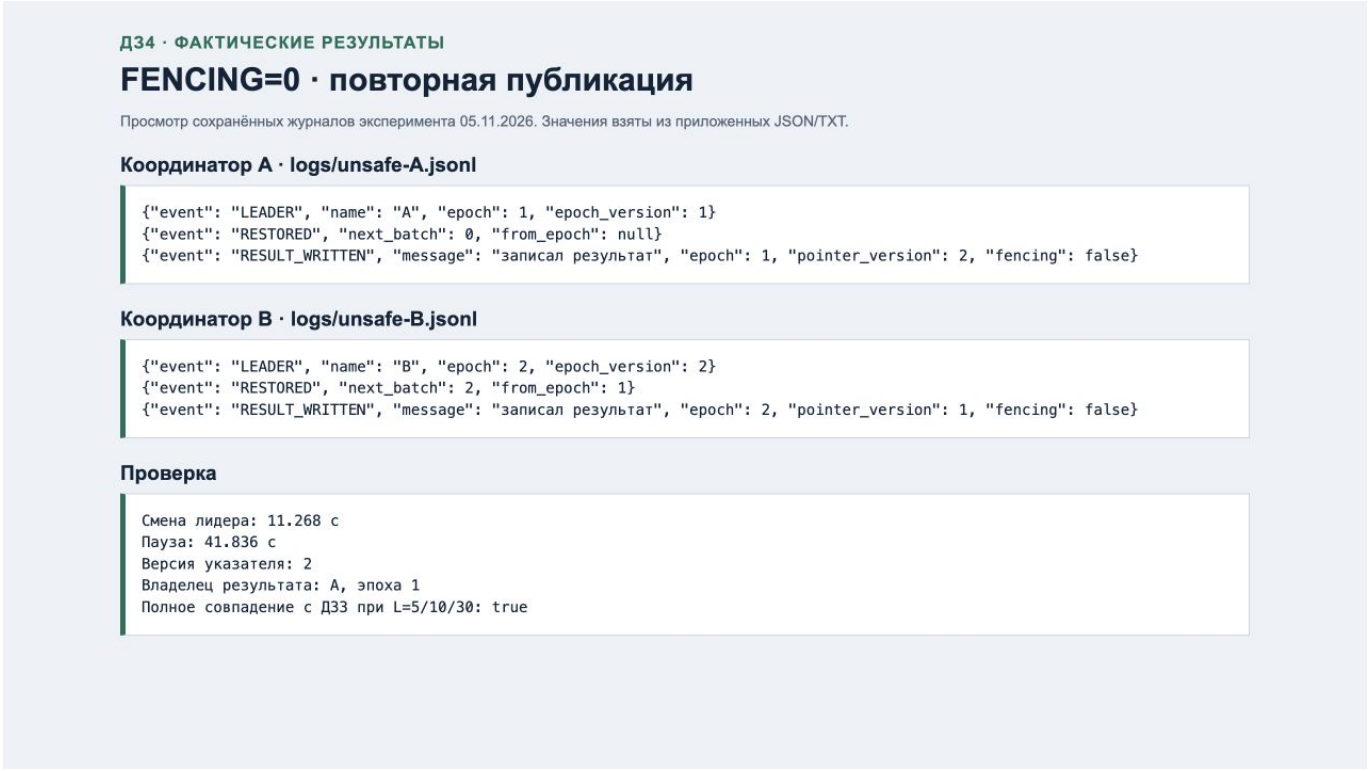
Чекпоинт содержит next_batch, хеш входа и все три накопленных счётчика. Большие данные сохраняются в общей папке, а в ZooKeeper записывается маленький указатель. Файл записывается через временный файл, fsync и os.replace. Контрольная сумма проверяется перед восстановлением. Каждая копия имеет собственные имена файлов; публикация чекпоинта защищена атомарной проверкой эпохи.

Сценарий	Восстановлен next_batch	Вакансий уже учтено	Эпоха снимка
unsafe	2	600	1
fenced	2	600	1

Новый лидер повторно читает детерминированный вход, пропускает уже посчитанные чанки и продолжает со следующего. Независимый верификатор пересчитал префикс данных и проверил все шесть восстановленных счётчиков. Поэтому доказано восстановление накопленных частот, а не только чтение номера чанка.

Функции batches и mapper из original_hw3/src/mapreduce.py не изменены. Вместо повторного скачивания из S3 используется идентичный подготовленный all_sources.csv; SHA-256 совпадает с Д33. Задержка 0.75 с после чанка нужна только для наблюдения отказа.

4. Остановка лидера без защиты



ДЗ4 · ФАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ

FENCING=0 · повторная публикация

Просмотр сохранённых журналов эксперимента 05.11.2026. Значения взяты из приложенных JSON/TXT.

Координатор A · logs/unsafe-A.jsonl

```
{"event": "LEADER", "name": "A", "epoch": 1, "epoch_version": 1}
{"event": "RESTORED", "next_batch": 0, "from_epoch": null}
{"event": "RESULT_WRITTEN", "message": "записал результат", "epoch": 1, "pointer_version": 2, "fencing": false}
```

Координатор B · logs/unsafe-B.jsonl

```
{"event": "LEADER", "name": "B", "epoch": 2, "epoch_version": 2}
{"event": "RESTORED", "next_batch": 2, "from_epoch": 1}
{"event": "RESULT_WRITTEN", "message": "записал результат", "epoch": 2, "pointer_version": 1, "fencing": false}
```

Проверка

```
Смена лидера: 11.268 с
Пауза: 41.836 с
Версия указателя: 2
Владелец результата: A, эпоха 1
Полное совпадение с ДЗ3 при L=5/10/30: true
```

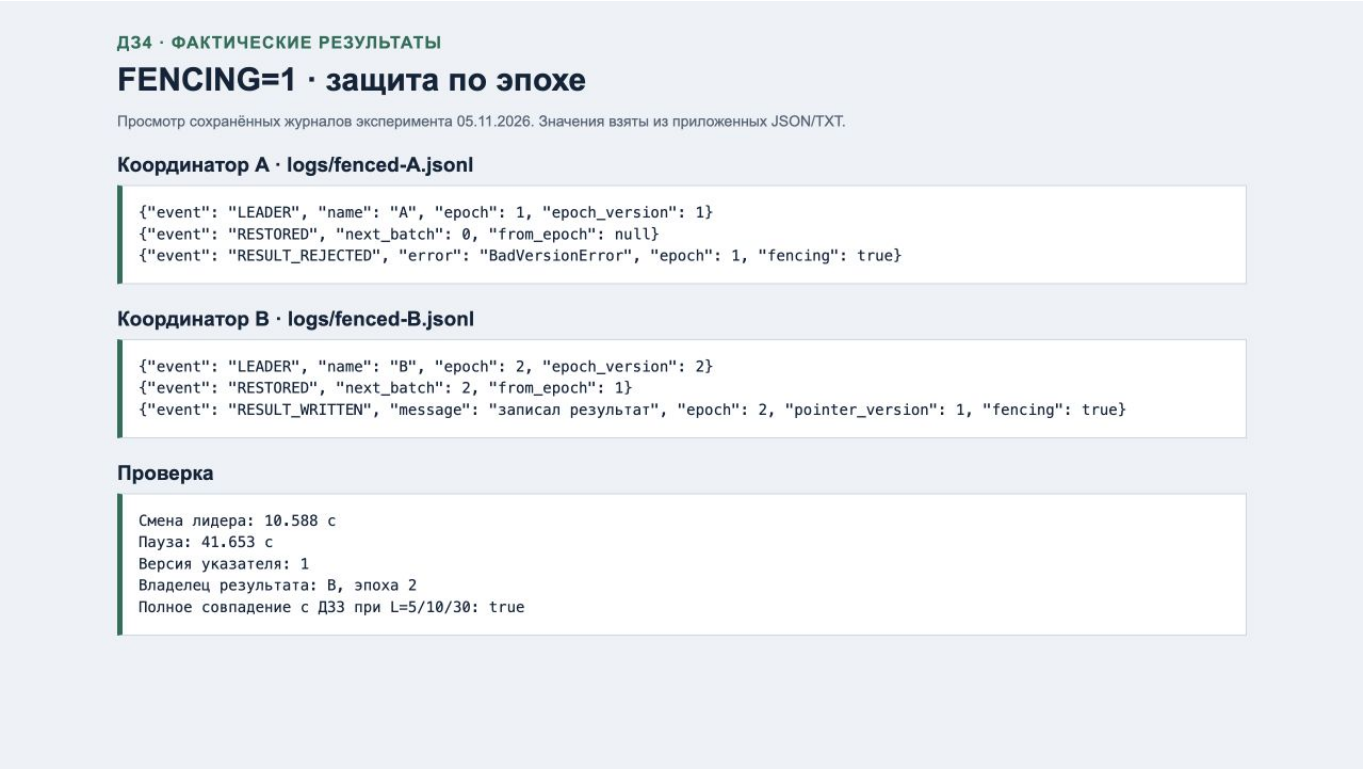
Скриншот браузера: локальная страница просмотра сохранённых результатов. Это представление фактических JSON/TXT, не снимок терминала. Полные журналы приложены.

Координатор A остановлен SIGSTOP после сохранения чекпоинта. Через 11.268 с B записал событие LEADER с эпохой 2. Он восстановился с next_batch=2, завершил расчёт и опубликовал результат. A оставался остановленным 41.836 с, до окончания работы B, после чего получил SIGCONT.

Старая копия A продолжила свой расчёт с захваченной эпохой 1. Её устаревшие чекпоинты отклонены, но защита публикации в этом режиме выключена. Поэтому результат записан второй раз: сначала B (версия указателя 1), затем A (версия 2). В журналах есть ровно два события RESULT_WRITTEN с текстом «записал результат».

Итоговый указатель принадлежит A, epoch=1, dataVersion=2. Это демонстрирует риск старого лидера даже при численно одинаковых результатах. Источники: logs/unsafe-A.jsonl, unsafe-B.jsonl; results/unsafe/summary.json.

5. Остановка лидера с FENCING=1



ДЗ4 · ФАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ

FENCING=1 · защита по эпохе

Просмотр сохранённых журналов эксперимента 05.11.2026. Значения взяты из приложенных JSON/TXT.

Координатор A · logs/fenced-A.jsonl

```
{"event": "LEADER", "name": "A", "epoch": 1, "epoch_version": 1}
{"event": "RESTORED", "next_batch": 0, "from_epoch": null}
{"event": "RESULT_REJECTED", "error": "BadVersionError", "epoch": 1, "fencing": true}
```

Координатор B · logs/fenced-B.jsonl

```
{"event": "LEADER", "name": "B", "epoch": 2, "epoch_version": 2}
{"event": "RESTORED", "next_batch": 2, "from_epoch": 1}
{"event": "RESULT_WRITTEN", "message": "записал результат", "epoch": 2, "pointer_version": 1, "fencing": true}
```

Проверка

```
Смена лидера: 10.588 с
Пауза: 41.653 с
Версия указателя: 1
Владелец результата: B, эпоха 2
Полное совпадение с ДЗ3 при L=5/10/30: true
```

Скриншот браузера: локальная страница просмотра сохранённых результатов. Это представление фактических JSON/TXT, не снимок терминала. Полные журналы приложены.

Повторён тот же сценарий с проверкой эпохи у получателя. Время от SIGSTOP до LEADER резервной копии: 10.588 с. Пауза A: 41.653 с. B восстановился с next_batch=2, получил epoch=2 и опубликовал результат.

После SIGCONT A сохранил свою исходную эпоху и версию. Получатель попытался атомарно проверить эту версию и обновить указатель; проверка завершилась BadVersionError. Запись отклонена, в журнале A есть RESULT_REJECTED, а RESULT_WRITTEN только один - у B. Итоговый указатель сохранил owner=B, epoch=2, dataVersion=1.

Первичная проверка выявила гонку SessionExpiredError при переподключении старой копии. Добавлено ограниченное повторение транзакции после восстановления сессии без обновления старой эпохи. Итоговые испытания проведены повторно на исправленном коде. Исходная диагностика сохранена в logs/diagnostics/.

6. Совпадение с результатами ДЗ3

В обоих сценариях обе копии завершили арифметический расчёт. Сравнение выполнено по всем ключам и частотам: 2 сценария × 2 копии × 3 размера стека = 12 полных словарей. Эталоны ДЗ3 дополнительно проверены независимым прямым перебором сочетаний, без импорта исходного mapper.

L	Размер сочетания	Уникальных	Сумма частот
5	1	924	35 509
5	2	6 698	67 318
5	3	14 424	64 806
10	1	1 416	52 837
10	2	18 585	181 102
10	3	88 454	397 529
30	1	1 714	60 254
30	2	33 821	285 780
30	3	300 547	1 157 213

Все девять сумм независимо совпали с Σ C(min(m,L),k), где m - число нормализованных навыков вакансии, k=1,2,3. На полном стеке: 33 821 уникальная пара, 285 780 вхождений пар; mysql + php = 3 522, git + php = 2 819, javascript + php = 2 622.

Контрольные суммы

Подготовленный вход ДЗ3:

`6a7708659dcc8923bf834a13a34e6a350f02ebec7fdf0f2b6219d28651910280`

Канонический результат L=5, одинаковый у всех четырёх вычислителей:

`84c1a03a8ab4c8ad155358bb6ccc7690a003a2b062cb33ab60dc1b25a7195c54`

Канонический результат L=10, одинаковый у всех четырёх вычислителей:

`8ba415e142ddeb5bf62676a8e8d2dc88242a9eda5d6551c234778057f4a7782c`

Канонический результат L=30, одинаковый у всех четырёх вычислителей:

`fae6112ea2c998ac47347857855d894b1c3fa9a48c8336308ccb74ec5cd42eba`

Хеши в отчёте относятся к каноническому JSON: UTF-8, sort_keys=True, без лишних пробелов. Файлы reference/ сохранены в прежнем формате, поэтому их байтовые хеши могут отличаться при равенстве всех данных. Подробная независимая проверка: results/validation.json и logs/validation.txt.

7. Ответы на вопросы

Почему три сервера, а не два и не четыре?

Запись должна быть подтверждена строгим большинством. Для N участников кворум равен floor(N/2)+1. Два сервера требуют оба голоса: отказ одного лишает систему кворума. Три требуют два голоса и переживают отказ одного. Четыре требуют три голоса, поэтому тоже переживают только один отказ, но расходуют больше ресурсов. Три - минимальное число для устойчивости к одному отказу; для двух отказов нужны пять. Пересечение любых двух большинств не позволяет независимым меньшинствам подтвердить несовместимые истории.

Серверов	Кворум	Допустимо отказов
2	2	0
3	2	1
4	3	1
5	3	2

Почему эпоху проверяет получатель записи, а не ZooKeeper?

ZooKeeper видит сессии, znodes и их версии, но не знает смысла сторонней записи: например, обновления файла, строки БД или отправки платежа. Приостановленный процесс не узнаёт вовремя о потере лидерства и может уже держать подготовленный запрос. Удаление его эфемерного узла само по себе не отменяет такой запрос. Получатель ресурса должен сравнить токен с актуальной эпохой непосредственно в момент применения записи.

В этом стенде получатель Receiver использует ZooKeeper как хранилище указателя, поэтому реализует проверку средствами версии znode и multi. Политика «проверять epoch» находится в коде получателя; ZooKeeper обеспечивает только атомарность и CAS. При переносе результата во внешнюю БД проверку нужно выполнять в транзакции этой БД. Отдельная проверка в ZooKeeper, а затем запись в БД оставила бы окно гонки.

```
tx = zk.transaction()
if fencing:
    tx.check(base + "/epoch", version=captured_epoch_version)
tx.set_data(base + "/result", pointer, version=-1)
# Один atomic commit; устаревшая эпоха -> BadVersionError.
```

Непринятый файл старого вычислителя может существовать как артефакт, но не становится опубликованным результатом: единственная точка публикации - защищённый указатель /result.

8. Воспроизведение и состав сдачи

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
docker compose -f infra/compose.yaml up -d
.venv/bin/python src/run_lab.py > logs/acceptance.txt 2>&1
.venv/bin/python src/verify_submission.py > logs/validation.txt
# Остановить стенд, сохранив тома:
docker compose -f infra/compose.yaml stop
```

Что проверять	Где находятся доказательства
Роли, Java, версия, команды	logs/commands.txt; results/infrastructure.json
Эфемерная сессия	logs/zkcli-ephemeral.txt
Остановка и смена лидера	results/{unsafe,fenced}/timeline.json и summary.json
Две записи / отказ по эпохе	logs/{unsafe,fenced}-{A,B}.jsonl
Серверные журналы	logs/servers.txt
Совпадение с ДЗ3	reference/; results/*/result-*.json
Чекпоинты восстановления	results/*/checkpoint-*.json
Независимая проверка	src/verify_submission.py; results/validation.json
Конфигурации	infra/compose.yaml; infra/zk1..3/{zoo.cfg,myid}
Скриншоты и исходные страницы	figures/*.png; report/evidence-*.html
Целостность комплекта	SHA256SUMS.txt; проверка ZIP CRC

Границы эксперимента

Это учебный стенд на одном Mac: три контейнера и общая папка не защищают от отказа самого компьютера. Остановка процессов, потеря сессии, смена лидера и проверка публикации были реальными. Старый процесс намеренно продолжает вычисления после потери сессии для демонстрации fencing. В рабочей системе дополнительно прекращают вычисления после LOST. Задержка искусственно растягивает расчёт, поэтому это измерение failover, а не производительности MapReduce. Оценка выставляется преподавателем; публикация в GitHub по просьбе пользователя не выполнялась.

Документация

Apache ZooKeeper 3.9.6: Programmer’s Guide (сессии, ephemeral, версии); Administrator’s Guide (ансамбль, кворум). Kazoo: Lock recipe. Использованная версия клиента 2.10.0 зафиксирована в requirements.txt.